

COMPLETE LEARNING ROADMAP

System Design

From first principles to planet-scale architecture

Scalability, reliability, availability — how the world's largest systems are designed and built

7 Phases · 50+ Concepts · 20 Case Studies

7	50+	20	~6mo	∞
PHASES	CONCEPTS	CASE STUDIES	TIMELINE	SCALE

PHASE 0 The Discipline of System Design

What it is, why it matters, and how to think about scale

What Is System Design?

System design is the process of defining the architecture, components, modules, interfaces, and data flow of a system to satisfy given requirements. Unlike coding, which is about correctness, system design is about trade-offs — every decision involves balancing competing concerns.

It sits at the intersection of software engineering, distributed systems, databases, networking, and operations. A senior engineer who designs systems well is one of the most valuable people in any technology organisation.

The Core Trade-Off Axes

Trade-Off	What It Means
Performance vs Cost	Faster systems cost more. Caching, CDNs, and SSDs improve performance but add expense.
Consistency vs Availability	CAP theorem: in the presence of network partitions, you must choose one. Most systems choose availability.
Latency vs Throughput	High throughput systems batch work; low-latency systems process immediately. Optimising both is hard.
Scalability vs Simplicity	Simple systems are easy to reason about. Scalable systems add layers that can fail.
Durability vs Speed	Writing to disk is durable but slow. In-memory is fast but volatile. Most systems layer both.
Flexibility vs Reliability	Flexible systems can be misused. Rigid contracts prevent certain bugs but limit evolution.
Read vs Write Optimisation	A system optimised for reads (denormalised, cached) will struggle with writes, and vice versa.
Centralisation vs Distribution	Centralised systems are simple; distributed systems are fault-tolerant but complex.

The System Designer's Mindset

There is no universally correct architecture. There are only architectures that are more or less appropriate for a given set of requirements, constraints, and scale. When you design systems, always start with: What are the requirements? What is the expected scale? What can we sacrifice? The answer to those questions determines everything else.

PHASE 1 Foundations

Networking, protocols, and how the web actually works

Core Topics — Weeks 1 to 3

You cannot design systems you don't understand at the protocol level. This phase covers the networking and protocol foundations that every system is built on.

The OSI & TCP/IP Models Seven layers from physical to application. Where each protocol lives. How data flows from your code to the wire.	TCP vs UDP TCP: reliable, ordered, connection-based. UDP: fast, fire-and-forget. When to choose each.
DNS — Domain Name System How names resolve to IPs. Authoritative vs recursive resolvers, TTLs, A/AAAA/CNAME records. DNS as a load balancer.	HTTP/1.1, HTTP/2, HTTP/3 From sequential requests to multiplexed streams to QUIC over UDP. Header compression, server push, connection coalescing.
TLS/SSL — Transport Security Symmetric vs asymmetric encryption, handshake, certificates, certificate authorities. Why HTTPS matters for system design.	REST, GraphQL & gRPC REST: resource-oriented, stateless, wide tooling. GraphQL: flexible queries, over-fetching solved. gRPC: binary, typed contracts, streaming.
WebSockets & SSE Long-lived connections for real-time data. WebSockets: bidirectional. SSE: server push only. Trade-offs for chat, live feeds, notifications.	IP Addressing & Subnets IPv4/IPv6, CIDR notation, private vs public ranges, NAT. Essential for VPC and cloud network design.
Load Balancing Algorithms Round robin, least connections, IP hash, weighted. Layer 4 vs Layer 7 load balancing. Health checks.	CDN — Content Delivery Networks Edge caching, origin pull vs push, cache invalidation. How CDNs reduce latency globally and absorb traffic spikes.

Phase 1 Projects

☐	Protocol Analyser Use Wireshark or tcpdump to capture and inspect real network traffic. Trace a full HTTP request — DNS lookup, TCP handshake, TLS negotiation, HTTP exchange. Build intuition for what happens at every layer. <i>Concepts: TCP/IP · DNS · HTTP · TLS</i>
☐	Design a CDN Architecture

On paper or in a diagram tool: design a CDN for a media streaming platform. Decide on edge locations, origin architecture, cache-control strategy, and purge mechanism.

Concepts: CDN · caching · DNS · load balancing

PHASE 2 Scalability Fundamentals

The mechanics of handling more load

Core Topics — Weeks 4 to 7

Vertical vs Horizontal Scaling

Vertical: bigger machine — simple but hits a ceiling.
Horizontal: more machines — complex but unlimited.
Most systems do both.

Caching — Layers & Strategies

L1/L2 CPU cache, in-process cache, distributed cache (Redis/Memcached). Cache-aside, read-through, write-through, write-behind. TTL vs eviction.

Database Read Replicas

One primary for writes, many replicas for reads.
Replication lag. When to use eventual vs synchronous replication.

Connection Pooling

Why opening a new DB connection per request kills performance. Pool sizing, queue depth, timeout strategy.

Autoscaling

Metric-based scaling (CPU, latency, queue depth).
Scale-out lag. Warm-up time. Predictive vs reactive scaling.

Stateless vs Stateful Services

Stateless services can scale horizontally trivially.
Stateful services need sticky sessions or externalised state. Design for statelessness.

Cache Invalidation

The second hardest problem in CS. Time-based expiry, event-driven invalidation, cache versioning.
Thundering herd problem.

Database Sharding

Horizontal partitioning across multiple databases. Hash sharding, range sharding, directory sharding. Hot spots and resharding.

Rate Limiting & Throttling

Token bucket, leaky bucket, fixed window, sliding window algorithms. Protecting services from abuse and cascading failure.

Geographic Distribution

Multi-region active-passive vs active-active. Data residency requirements. Latency-based routing.

Phase 2 Projects



Scale a Monolith

Start with a single-server architecture for a Twitter-like app. Systematically identify bottlenecks: DB reads, session state, static assets. Apply caching, replicas, and CDN. Document each trade-off.

Concepts: vertical scaling · read replicas · caching · stateless design



Design a Rate Limiter

Design a distributed rate limiter service. Choose an algorithm (sliding window log recommended). Decide on storage (Redis), handle distributed counting across nodes, and plan for Redis failure.

Concepts: rate limiting · Redis · sliding window · distributed counters

PHASE 3 Databases & Storage*Choosing and designing the right storage for every problem***Core Topics — Weeks 8 to 11****Relational Databases (RDBMS)**

Tables, rows, ACID transactions, normalisation, foreign keys, joins, indexes. PostgreSQL and MySQL internals: B-tree indexes, WAL, MVCC.

NoSQL — Key-Value Stores

Redis, DynamoDB. $O(1)$ lookups. Ideal for sessions, caches, leaderboards, counters. Data structures beyond strings.

NoSQL — Graph Databases

Neo4j, Neptune. Nodes and edges. Ideal for social graphs, recommendation engines, fraud detection.

CAP Theorem

Consistency, Availability, Partition Tolerance — pick two. CP systems (strong consistency) vs AP systems (always available). Understanding what your system needs.

Time-Series Databases

InfluxDB, TimescaleDB. Append-only writes, downsampling, retention policies. Purpose-built for metrics and IoT.

Data Warehouses

Snowflake, BigQuery, Redshift. Columnar storage, analytical queries, OLAP vs OLTP. When to move data out of your operational DB.

NoSQL — Document Stores

MongoDB, DynamoDB. Schema-flexible, denormalised, nested documents. When normalisation costs more than it saves.

NoSQL — Wide-Column Stores

Cassandra, HBase. Optimised for time-series, high-write workloads. Partition key design determines everything.

ACID vs BASE

ACID: Atomicity, Consistency, Isolation, Durability — strong guarantees. BASE: Basically Available, Soft state, Eventually consistent — higher availability.

Database Indexing Deep Dive

B-tree, hash, composite, covering, partial indexes. Index selectivity. The cost of write amplification. EXPLAIN ANALYSE.

Object Storage

S3, GCS, Azure Blob. Flat namespace, no hierarchy, eventual consistency. Pre-signed URLs, lifecycle policies. Not a filesystem.

Database Selection Framework

Ask: read/write ratio, consistency requirements, query patterns, scale, team expertise, cost. There is no universally best database.

Phase 3 Projects**Design a Multi-Model Storage Layer**

Design the complete storage architecture for Instagram: user profiles (RDBMS), posts/media (object storage), feed (Redis sorted sets), social graph (graph DB), analytics (data warehouse). Justify each choice.

Concepts: RDBMS · Redis · object storage · graph DB · warehouse

**Cassandra Schema Design**

Design a Cassandra schema for a ride-sharing apps trip history. Model around query patterns (not relationships). Design partition keys to avoid hot spots. Compare with a PostgreSQL schema for the same data.

Concepts: Cassandra · partition keys · wide-column · query-driven design

PHASE 4 Distributed Systems*Consensus, replication, messaging, and the hardest problems in computing***Core Topics — Weeks 12 to 17****Distributed System Challenges**

Partial failures, network partitions, clock skew, message reordering. The fallacies of distributed computing. Why distributed is hard.

Consensus Algorithms

Paxos and Raft. How distributed systems agree on a single value. Leader election, log replication, safety vs liveness.

Message Queues & Brokers

Kafka, RabbitMQ, SQS. Decoupling producers from consumers. At-most-once, at-least-once, exactly-once delivery semantics.

Event-Driven Architecture

Events as the primary communication mechanism. Event sourcing, CQRS. Decoupled services that react to things that happened.

Idempotency

Designing operations that can be safely retried. Idempotency keys. Why at-least-once delivery requires idempotent consumers.

Vector Clocks & Causality

Tracking causality in distributed events. Happens-before relationship. How DynamoDB handles concurrent writes.

Replication Strategies

Single-leader, multi-leader, leaderless (Dynamo-style). Synchronous vs asynchronous replication. Failover and split-brain.

Consistent Hashing

Distributing data across nodes so that adding/removing a node only remaps a fraction of keys. Virtual nodes to handle heterogeneous hardware.

Apache Kafka In Depth

Topics, partitions, offsets, consumer groups. Log compaction. Exactly-once semantics. Kafka as an event store vs a queue.

Distributed Transactions

Two-phase commit (2PC), saga pattern, outbox pattern. How to maintain consistency across services without a distributed lock.

Distributed Locking

Redis SETNX, Redlock algorithm, ZooKeeper locks. When you need mutual exclusion across services. Pitfalls and alternatives.

Gossip Protocols

How nodes in a cluster discover each other and share state. Used in Cassandra, Redis Cluster. Epidemic algorithms.

Phase 4 Projects**Design a Message Queue**

Design a durable, at-least-once message queue from first principles. Design the producer API, storage format, consumer group model, acknowledgement mechanism, and dead-letter queue.

Concepts: [message queues](#) · [durability](#) · [consumer groups](#) · [at-least-once](#) · [DLQ](#)

**Design a Distributed Key-Value Store**

Design a system like DynamoDB or Cassandra: consistent hashing for sharding, quorum reads/writes for tunable consistency, gossip for membership, vector clocks for conflict resolution.

Concepts: consistent hashing · quorum · replication · gossip · vector clocks

PHASE 5 Microservices & API Design*Decomposing systems, service communication, and contracts***Core Topics — Weeks 18 to 22****Monolith vs Microservices**

Monolith: simple, fast to develop, hard to scale independently. Microservices: independently deployable, independently scalable, operationally complex. The right choice depends on team size and maturity.

API Gateway Pattern

Single entry point for all clients. Authentication, rate limiting, routing, protocol translation, response aggregation. Kong, AWS API Gateway.

Inter-Service Communication

Synchronous (REST/gRPC) vs asynchronous (events/queues). Choosing based on coupling, latency requirements, and failure modes.

Bulkhead Pattern

Isolate failures to a subset of the system. Thread pool isolation, resource quotas per service. The Titanic had bulkheads — microservices should too.

API Versioning

URI versioning, header versioning, query param versioning. Semantic versioning for APIs. How to evolve APIs without breaking clients.

Service Decomposition

How to split a monolith. Domain-driven design (DDD): bounded contexts, aggregates, domain events. Decompose by business capability, not by technical layer.

Service Discovery

How services find each other without hardcoded IPs. Client-side (Eureka) vs server-side (ALB) discovery. Service registries. DNS-based discovery.

Circuit Breaker Pattern

Fail fast when a downstream service is struggling. Closed, open, half-open states. Hystrix, Resilience4j. Preventing cascading failures.

Sidecar & Service Mesh

Istio, Linkerd, Envoy. Offload cross-cutting concerns (mTLS, retries, tracing, circuit breaking) into a network proxy alongside every service.

Contract Testing

Consumer-driven contract tests (Pact). Ensuring that provider changes don't break consumers without full integration test suites.

Phase 5 Projects**Decompose a Monolith**

Take an e-commerce monolith (users, products, orders, payments, notifications). Identify bounded contexts. Design the service boundaries, communication patterns, and migration strategy using Strangler Fig.

Concepts: DDD · bounded contexts · strangler fig · service boundaries · async messaging

**Design an API Gateway**

Design a production API gateway: authentication (JWT validation), per-user rate limiting, request routing, response caching, protocol translation (REST to gRPC), and observability. Plan for 100k req/sec.

Concepts: API gateway · JWT · rate limiting · caching · routing · observability

PHASE 6 Reliability, Observability & Security*Building systems that survive the real world***Core Topics — Weeks 23 to 27****High Availability Patterns**

Active-passive failover, active-active multi-region, N+1 redundancy. SLAs vs SLOs vs SLIs. Measuring and targeting availability.

Disaster Recovery

RTO vs RPO. Backup strategies: full, incremental, differential. Pilot light, warm standby, hot standby. Regular DR drills.

Alerting & On-Call

SLO-based alerting vs symptom-based. Alert fatigue. Runbooks. Incident response process. Blameless post-mortems.

Security — Authentication

Session tokens, JWTs, OAuth 2.0, OIDC. API keys. mTLS for service-to-service. The difference between authentication and authorisation.

Security — Data Protection

Encryption at rest and in transit. Key management (KMS, Vault). Data classification. GDPR implications for system design. PII handling.

Fault Tolerance

Design for failure. Retry with exponential backoff and jitter. Timeouts. Graceful degradation. Chaos engineering (Netflix Chaos Monkey).

Metrics, Logging & Tracing

The three pillars of observability. Metrics (Prometheus/Grafana), structured logs (ELK stack), distributed traces (Jaeger/Zipkin). Correlation IDs.

Distributed Tracing

How a single request flows through many services. Spans and traces. OpenTelemetry. Sampling strategies. Finding the slow service.

Security — Authorisation

RBAC, ABAC, PBAC. OPA (Open Policy Agent). Centralised vs decentralised policy enforcement. Principle of least privilege.

DDoS Mitigation

Anycast routing, scrubbing centres, WAF rules, challenge pages, IP reputation lists. Cloudflare, AWS Shield. Rate limiting as a first line of defence.

Phase 6 Projects**Design an Observability Stack**

Design a complete observability platform for a 50-microservice system: metrics collection, log shipping and indexing, distributed trace ingestion, dashboard design, and SLO-based alerting.

Concepts: Prometheus · Grafana · ELK · Jaeger · OpenTelemetry · SLOs

**Design an Auth System**

Design the authentication and authorisation for a multi-tenant SaaS platform: OAuth 2.0 with PKCE, JWT issuance and validation, refresh token rotation, role-based access with tenant isolation.

Concepts: OAuth 2.0 · JWT · RBAC · multi-tenant · audit logs

PHASE 7 Real-World Case Studies*How the world's largest systems are actually designed*

20 Systems to Design

The best way to learn system design is to design systems. These 20 case studies cover the full breadth of concepts. For each one: gather requirements, estimate scale, design the data model, choose components, draw the architecture, then identify bottlenecks.

System	Key Concepts to Cover
URL Shortener (e.g. bit.ly)	Key concepts: hashing, 301 vs 302, hot URLs, analytics, KV store
Social Media Feed (e.g. Twitter)	Key concepts: fanout-on-write vs read, timeline, ranking, celebrity problem
Search Autocomplete	Key concepts: trie, prefix trees, ranking, low latency, offline computation
Distributed Cache (e.g. Redis)	Key concepts: eviction policies, consistent hashing, replication, pipelining
Chat Application (e.g. WhatsApp)	Key concepts: WebSockets, message ordering, E2E encryption, presence, read receipts
Video Streaming (e.g. YouTube)	Key concepts: CDN, adaptive bitrate, chunking, transcoding pipeline, recommendation
Ride-Sharing (e.g. Uber)	Key concepts: geo-indexing (S2 library), matching, surge pricing, real-time tracking
Web Crawler	Key concepts: politeness, distributed BFS, deduplication, robots.txt, storage
Notification System	Key concepts: fan-out, templating, multi-channel (push/email/SMS), priority queues
Rate Limiter	Key concepts: token bucket, sliding window, Redis, distributed counters, bypass list

System	Key Concepts to Cover
Payment Processing System	Key concepts: idempotency, saga pattern, ledger design, PCI compliance, fraud
Photo Sharing (e.g. Instagram)	Key concepts: media storage, feed generation, follow graph, explore, sharding

Cloud File Storage (e.g. Dropbox)	Key concepts: chunking, delta sync, versioning, conflict resolution, quotas
Hotel Booking (e.g. Booking.com)	Key concepts: inventory locking, overbooking, ACID transactions, search
Ad Click Aggregator	Key concepts: stream processing, approximate counting, Flink/Spark, privacy
Collaborative Doc Editor (e.g. Docs)	Key concepts: operational transforms or CRDTs, presence, revision history
Metrics & Monitoring System	Key concepts: time-series DB, aggregation, alerting, cardinality explosion
Live Leaderboard	Key concepts: sorted sets, real-time updates, eventual consistency, sharding
E-Commerce Search	Key concepts: inverted index, Elasticsearch, ranking, autocomplete, facets
API Gateway	Key concepts: auth, routing, rate limiting, circuit breaking, observability

Interview Framework

Use this consistent structure for every system design interview or self-study session:

Step	What to Do
Step 1: Clarify Requirements	Functional requirements (what it does) and non-functional (scale, latency, availability, consistency). Ask before designing.
Step 2: Define Scale	DAU, requests/sec, data volume, storage, read/write ratio. Back-of-envelope calculations guide every decision.
Step 3: High-Level Design	Main components: clients, LBs, services, caches, databases, queues. Draw boxes and arrows before adding detail.
Step 4: Deep Dive	The interviewer will direct you. Pick the hardest part: DB schema, API design, caching strategy, or a specific algorithm.
Step 5: Identify Bottlenecks	Which component hits its limit first? How do you scale it? What breaks under failure? Propose solutions.
Step 6: Discuss Trade-offs	What did you sacrifice? What would you do differently with more time? What would you monitor first in production?

The Golden Rule of System Design

Requirements drive architecture. Never jump to a solution before you fully understand the problem. The same system built for 1,000 users vs 100 million users is a completely different system — even if the product features are identical.

System Design Roadmap · 7 Phases · ~6 Months